

ИНДЕКСЫ

Индексы — это специальные структуры данных, ускоряющие доступ к строкам таблиц по значениям столбцов. В PostgreSQL они особенно полезны для WHERE, JOIN, ORDER BY, GROUP BY и агрегаций. Есть разные типы: B-Tree, Hash, GiST, GIN и др. Индексы ускоряют выборки, но замедляют вставки/обновления/удаления.

Индекс представляет собой отсортированный список значений одного или нескольких столбцов, с указателями на соответствующие строки в таблице.

Индекс:

- Упорядочен (например, по возрастанию чисел или алфавиту);
- Ссылается на строки в таблице;
- Автоматически используется СУБД при выполнении запроса (если сочтет нужным).

Индекс применяется:

- При **фильтрации** (WHERE): WHERE StudID = 18;
- При **соединении таблиц** (JOIN ON): ON Students.GroupID = Groups.ID;
- При **агрегации** (MIN, MAX) — если по индексируемому столбцу;
- При **сортировке** (ORDER BY);
- При **группировке** (GROUP BY);
- При **поиске по шаблону**, если шаблон начинается с фиксированных символов (например, 'abc%', но не '%abc').

ТИПЫ ИНДЕКСОВ

1. B-tree (по умолчанию)

Стандартный тип. Используется для сравнения (=, <, >, BETWEEN, ORDER BY, LIKE 'abc%').

2. Hash-индекс

Для точных сравнений =. Не поддерживает диапазоны.

3. GiST (Generalized Search Tree)

Для гео-данных, полнотекстового поиска, вложенных диапазонов и пр.

4. GIN (Generalized Inverted Index)

Для массивов, JSONB, документов, полнотекстового поиска (@@, tsvector).

5. SP-GiST

Для иерархий, IP-сетей, кластеров (быстрый поиск в редких структурах).

6. BRIN (Block Range Index)

Для очень больших таблиц, где значения идут по порядку. Мал по размеру, но ограничен.

Когда индексы неэффективны

- **Таблица маленькая** — проще просканировать всю таблицу.
- **Выборка большая** (например, WHERE age > 0) — индекс не даст выгоды.
- **Шаблон начинается с %** — индекс LIKE '%abc' не используется.
- **После сильных обновлений** — индекс может стать "грязным", и СУБД выберет полный перебор.

Когда СУБД сама не использует индекс

- Если статистика считает, что выгоднее полный перебор;
- Если WHERE содержит OR с неиндексируемыми условиями;
- Если соединение требует материализации и сортировки вне индекса.

Индексы и план выполнения запроса

СУБД:

1. Парсит запрос;
2. Строит дерево;
3. Смотрит на доступные индексы;
4. Выбирает **оптимальный план** (по оценке стоимости: I/O, CPU, объем вывода);
5. Может использовать индекс или проигнорировать его.

ПОДРОБНО ПРО КАЖДЫЙ ТИП

1. B-tree (по умолчанию)

Структура:

- Сбалансированное дерево поиска (B+Tree), где:
 - **Листовые узлы** содержат ключи и ссылки на строки таблицы;
 - **Внутренние узлы** — только ключи и указатели на дочерние узлы.

Работа:

- Данные **отсортированы**.
- Поиск осуществляется **двоичным спуском** от корня к листу (логарифмическая сложность).
- Листья связаны **двусвязным списком** — быстрая сортировка и сканирование по диапазону.

Поддерживает:

- =, <, >, BETWEEN, ORDER BY, LIKE 'abc%'
- может использоваться для DISTINCT, MIN(), MAX()

2. Hash-индекс

Структура:

- Хеш-таблица, в которой значения прогоняются через хеш-функцию, и результат указывает на сегмент ссылки на строки.

Работа:

- Быстрый поиск по точному совпадению =.
- Распределение по **хеш-бакетам**.
- **Не поддерживает** диапазонные операции.

Ограничения:

- Раньше (до PostgreSQL 10) не поддерживались транзакции и журнал WAL.
- Работает только с =, не может быть использован в ORDER BY.

3. GiST (Generalized Search Tree)



Структура:

- **Обобщенное дерево поиска**, гибко определяемое пользователем.
- Хранит не конкретные значения, а **приближения (bounding boxes, диапазоны)**.



Работа:

- Используется в гео-пространственных запросах, полнотекстовом поиске, вложенных диапазонах.
- Обеспечивает **приблизительный поиск** с возможностью уточнения на втором шаге.



Поддерживает:

- @>, <<, && и др. — например, для интервалов, координат, IP-диапазонов.

4. GIN (Generalized Inverted Index)



Структура:

- **Инвертированный индекс**: мапа «значение → список документов/строк», как в поисковых системах.



Работа:

- Очень эффективен для:
 - массивов (int[])
 - JSONB (@>, ?)
 - полнотекста (tsvector)
- Позволяет быстро находить все строки, где значение **входит в массив, JSON или текст**.



Поддерживает:

- @@, @>, ?, ?|, ?&, ~, и др.

5. SP-GiST (Space-partitioned GiST)



Структура:

- Делит пространство на части (например, квадранты).
- Подходит для **нераспределённых, иерархических структур** (IP-сети, дерево вложенности).



Работа:

- Использует **жесткую** структуру для разбиения значений (например, префикс дерева).
- Эффективен для **поиска ближайших соседей** и маршрутизации.

6. BRIN (Block Range Index)



Структура:

- Хранит **сводную информацию о блоках строк** (например, минимальное и максимальное значение).



Работа:

- Не индексирует каждую строку.
- СУБД проверяет, попадает ли условие запроса в **диапазон значений блока**.



Подходит только, если:

- Таблица **огромная**;
- Данные **отсортированы по индексу**.

СРАВНИТЕЛЬНАЯ ТАБЛИЦА

Тип индекса	Структура	Лучше всего для	Поддерживает	Особенности
B-tree	B+Tree	=, <, >, диапазоны	ORDER BY, WHERE	По умолчанию
Hash	Хеш-таблица	Только =	WHERE	Очень быстрый =, но ограничен
GiST	Обобщённое дерево	Гео, диапазоны	@>, && и т.п.	Приблизительный поиск
GIN	Инверт. индекс	Массивы, JSONB, текст	@>, @@, ? и др.	Идеален для contains и fulltext
SP-GiST	Пространственное	IP, маршруты	<<= и пр.	Иерархии, маршруты
BRIN	Блочный	Очень большие таблицы	Диапазоны	Маленький, но грубый